

COMP7710

Programming Fundamentals

Today's lecture
Git, GitLab, and IntelliJ

Course Convenor
Bernardo Pereira Nunes



Australian
National
University

All lectures are recorded and made available on Canvas

Agenda

- > Version Control
- > What is Git?
- > Basic Git Usage
- > IntelliJ and VCS
- > Exercises



Version Control

- > A system that **tracks changes** to files over time
- > Let you **save, view, and restore** previous versions
- > *Commonly used for source code, but works for any file (docs, images, designs)*

Why should you use it?

- > ***Undo*** mistakes and ***recover*** lost work
- > ***Compare versions*** and see what changed
- > ***Trace issues***: who changed what, when, and why
- > ***Revert*** a single file or an entire project
- > ***Low overhead, high payoff for teamwork and safety***



Local Version Control Systems

- > Common approach: **copy files into another folder**
- > Sometimes with timestamps
- > Simple and familiar

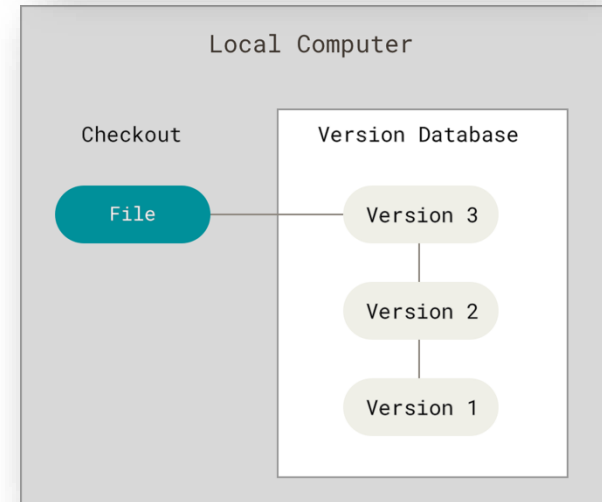
Why does this fail?

- > **Error-prone**
- > Easy to **edit the wrong copy**
- > Easy to **overwrite or lose files**
- > Hard to know **which version is the right one**

Is there a better solution?

- > **Local Version Control Systems (VCS)**
- > Use a **database** to track all file changes
- > **Safer, clearer, and designed for managing versions**

Source: <https://git-scm.com/book/en/v2/>

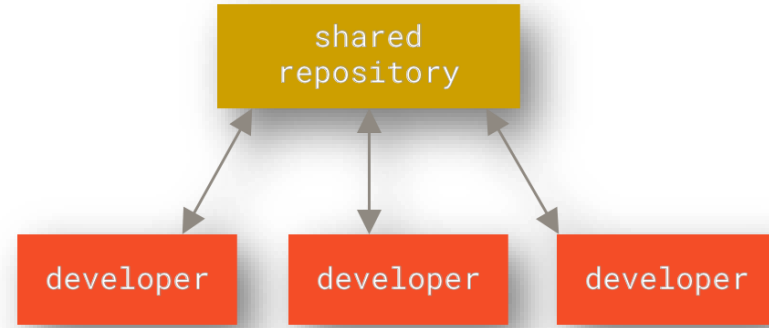


Local version control diagram



Centralised Version Control Systems

- > Centralised Version Control Systems (CVCS) support collaboration by maintaining a *single central repository that stores the complete version history of a project* (e.g., SVN)
- > **Developers interact with the system by checking out files from the central server and committing changes back to it**



Centralised version control diagram

Advantages

- > All changes are managed through a single authoritative source (centralised)
- > Users can **easily track project activity / contributions** (commit directly to the central server)
- > **Easier to manage than maintaining local repositories** on each client

Disadvantages

- > **Single point of failure:** if the central server becomes unavailable, collaboration halts
- > Server or disk failure without proper backups can result in **data loss**
- > Most operations require continuous access to the central server (**limited offline work**)



Distributed Version Control Systems

> *Distributed Version Control Systems (DVCS) replicate the entire repository, including its full history, to each client*

>> *Developers work with a **complete local copy** rather than a single shared snapshot*

> *Common DVCS include **Git**, Mercurial, and Darcs*

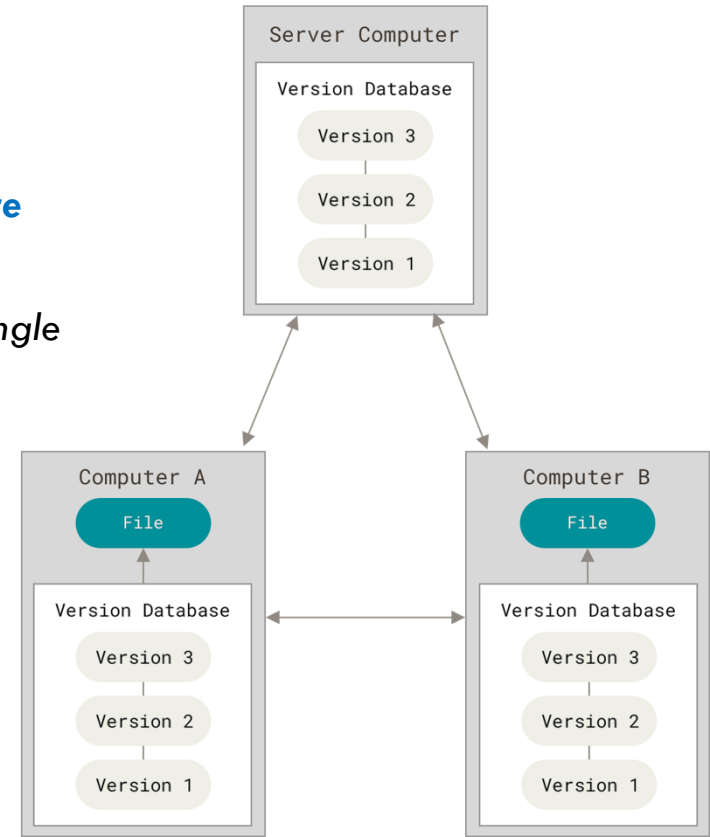
Key Characteristics

> **Every clone** contains the **complete project history**

> If a central or shared server fails, the repository can be fully restored from any client (**fault tolerant**)

> Most version control operations can be performed without network access (**offline support**)

> ***DVCS trade simplicity and central control for resilience, flexibility, and scalability***



Distributed version control diagram



GIT

- > Developed in 2005 by Linus Torvalds in response to the loss of free access to BitKeeper, previously used by the Linux kernel project
- > Designed to support the needs of large-scale, distributed software development

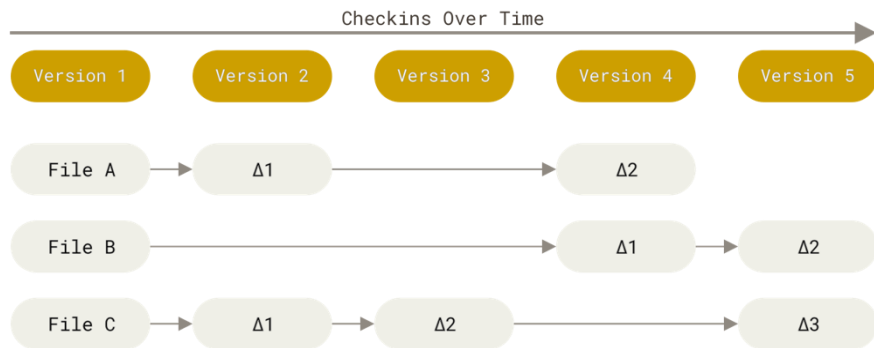
Design Goals

- > High performance (speed)
 - > Simple internal design
 - > Strong support for **non-linear development** (branching)
 - > Distributed architecture
 - > Efficient handling of very large codebases (e.g., Linux Kernel)
-
- > Git evolved into a mature, fast, and scalable DVCS, widely adopted for both open-source and industrial software development

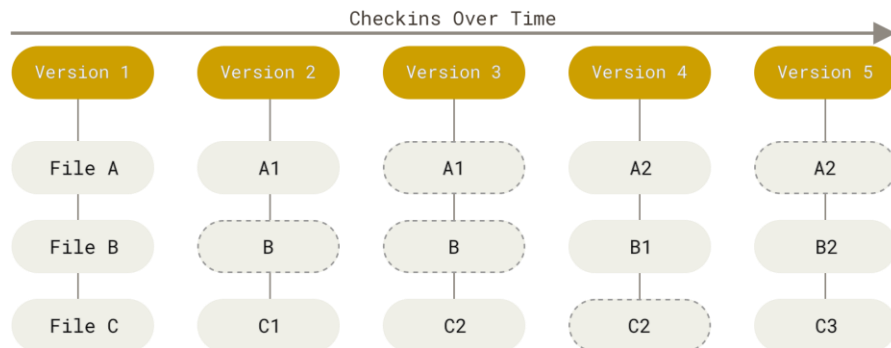


How GIT works?

- > Stores data as a sequence of **snapshots of the entire project state** (**not file diffs**)
- > Each commit represents a snapshot of a **miniature filesystem** at a specific point in time
- >> **Traditional version control systems** store history as **file-based changes (deltas)** over time. Each revision records how files have changed relative to a previous version
- > Unchanged files are not duplicated; Git stores references to existing objects for efficiency



Storing data as changes to a base version of each file



Storing data as snapshots of the project over time



How GIT works?

a short, fixed-length value computed from data and used to verify its integrity

> Git identifies and stores all data using cryptographic **checksums**, so any change or corruption is automatically detected

> It uses SHA-1 hashes (40-character hexadecimal strings) as content-based identifiers, meaning objects are **referenced by what they contain, not by name**

>> A SHA-1 hash looks something like this: 24b9da6552252987aa493b52f8696cd6d3b00373

computationally infeasible to reconstruct the original data from the hash

> Git has **three main states** that your files can reside in:

>> Modified: The **file has been changed** in the working directory but is **not yet recorded in the repository**

>> Staged: The modified file has been selected and **added to the staging area to be included in the next commit**

>> Committed: The file's current version is **permanently stored in the local Git repository**

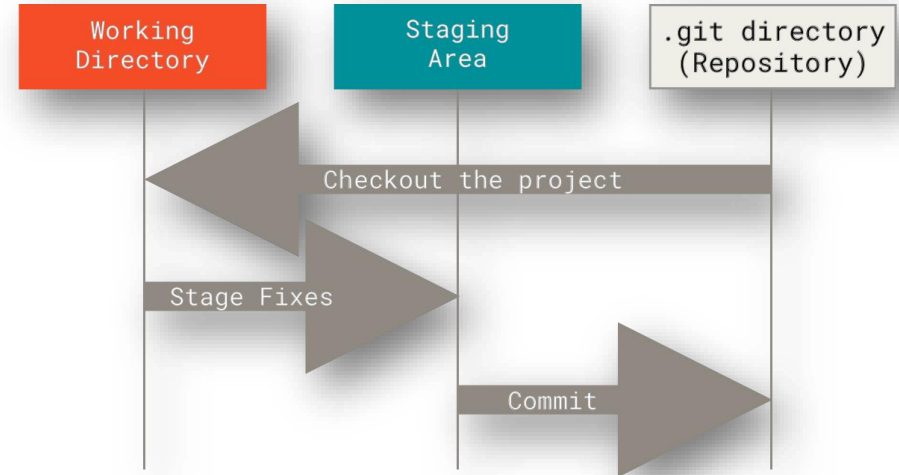


How GIT works?

- > Working Tree (Working Directory): A **local checkout of a specific project version**, where files are viewed, edited, and modified
- > Staging Area (Index): An intermediate area that **records which changes will be included in the next commit**
- > Git Directory (Repository): Stores the **complete object database and metadata of the project**; this is the core of Git and is what gets copied during cloning

Basic Git Workflow

- > Modify files in the working tree
- > Selectively add changes to the staging area
- > Commit the staged snapshot to the Git directory



Working tree, staging area, and Git directory



Installing Git

Before you start using Git, you must install it on your computer:

>> <https://git-scm.com/install/windows>

>> <https://git-scm.com/install/mac>

>> <https://git-scm.com/install/linux>

> Note that you can use the command `git --version` in the terminal to check whether Git is installed on your computer on Windows, macOS, or Linux

> **On macOS**, if Git is not installed, the system may prompt you to install the Command Line Developer Tools, which include Git

> **On Windows**, Git is not installed by default and must be installed manually

> **On Linux**, Git is usually installed via the system's package manager



Customising Git

Now that you have installed Git on your computer, you need to configure it. This configuration defines how Git looks and behaves.

Git has three logical configuration levels:

- > **System level** /etc/gitconfig: Applies to all users on the machine
- > **Global level (user)** ~/.gitconfig or ~/.config/git/config: Applies to all repositories for the current user
- > **Local level (repository)** .git/config: Applies only to that repository

> Precedence model: **local** overrides **global**, which overrides **system**

```
% git config --list --show-origin      (shows the exact files)
% git config --list --show-scope        (shows the logical level)
```

> We must now configure our git identity:

```
% git config --global user.name "Bernardo Nunes"
% git config --global user.email Bernardo.Nunes@anu.edu.au
```

This information is recorded in every commit and cannot be changed retroactively. Set your user.name and user.email immediately after installing Git



Customising Git

```
% git config --global user.name "Bernardo Nunes"  
% git config --global user.email Bernardo.Nunes@anu.edu.au
```

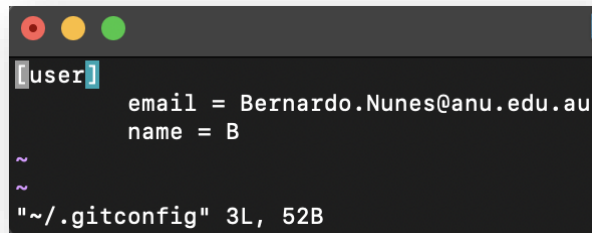
> If you end up with multiple `user.name` entries, Git will use the **last one it reads**. You may want to remove unused entries using the following command:

```
% git config --global --unset-all <key>
```

> Examples of keys include `user.name`, `user.email`, and `core.editor`

> You can also edit the configuration files directly.

>> Use: `git config --list --show-origin` to see which files define each setting and then use a text editor to edit the corresponding keys

A terminal window with a dark background and light text. The prompt is '[user]'. The output shows 'email = Bernardo.Nunes@anu.edu.au' and 'name = B'. Below this, there are two tilde '~' characters and a line showing '"~/ .gitconfig" 3L, 52B'.

```
[user] email = Bernardo.Nunes@anu.edu.au  
      name = B  
~  
~  
"~/ .gitconfig" 3L, 52B
```

> Use `git help <verb>` to access Git's built-in documentation. Use `git <verb> -h` for concise help.

>> Tools such as ChatGPT, DeepSeek, and Claude can also be helpful, but `git help` is always available when you do not have access to those tools.



Git Basics

Getting a Git Repository

- > You can obtain a Git repository in two main ways:
- >> Initialize a new repository from an existing local directory
- >> Clone an existing repository from a remote source

a local directory that is currently not under version control

In both cases, you end up with a local Git repository ready for development

Initializing a Repository in an Existing Directory

- > Navigate to the project directory (% `cd your-project`)
- > Use the command `git init` to initialize an empty git repository and create a new subdirectory named `.git` that contains all of your necessary repository files

<Quick Demo>

Let's quickly create a file, then run:

```
% git add <file>
% git status
% git rm --cached <file>
% git diff --staged
% git commit -m "message"
```

to add all modified and new files in the current directory and its subdirectories, use `git add .`

`git add -p <file>` stages changes interactively

You do not have to commit everything you changed in a file; you can choose the parts that make sense to include in a single commit. **Recommendation:** Make one logical change at a time, then stage and commit. Main options: y (yes), n (no), s (split), and, e (manually edit the hunk).

`git diff` compares what is in your working directory with what is in your staging area. "`--staged`" compares what you staged with your last commit.



Git Basics

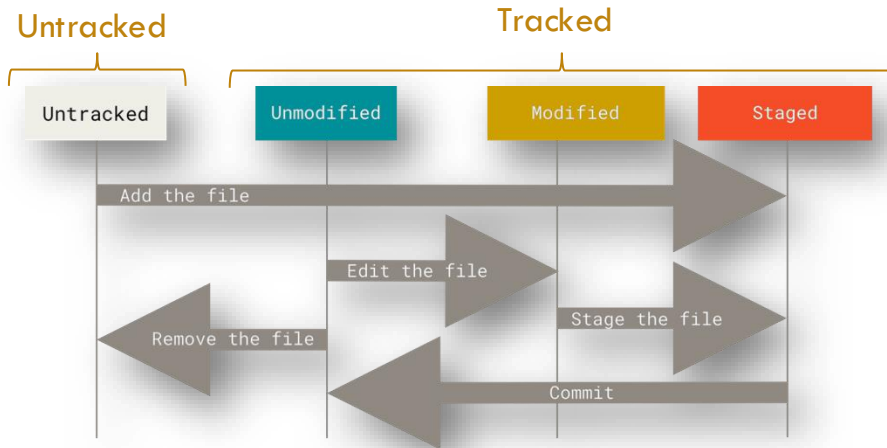
<Quick Demo>

Let's quickly create a file, then run:

```
% git add <file>
% git status
% git rm --cached <file>
% git diff --staged
% git commit -m "message"
```

Git add is a **multipurpose command**. You use it to **begin tracking new files**, to **stage files**, and to do other things like **marking merge-conflicted files as resolved**.

What happens if you run `git add README.md`, then edit `README.md` again and run `git status`? What happens if you commit at this point?



The lifecycle of the status of your files

-s: show short status

```
$ git status -s
M README
MM Rakefile
A lib/git.rb
M lib/simplegit.rb
?? LICENSE.txt
```

XY <file>

Left column (X) → staging area

Right column (Y) → working directory

_M README: file modified but not staged

?? → Untracked files

A → added to the staging area and ready to be committed

MM → modified and staged, and then modified again after staging



Git Basics

Git Ignore

> A .gitignore file tells Git which files or patterns to ignore, so they are not tracked or included in commits

Why would we want to ignore files?



- >> Files that may be the product of building your code
- >> Temporary files
- >> Logs
- >> Local configuration files

Additional info

```
$ git commit -m "Story 182: fix benchmarks for speed"
[master 463dc4f] Story 182: fix benchmarks for speed
2 files changed, 2 insertions(+)
create mode 100644 README
```



You can see that the commit produces output describing itself: the branch you committed to (master) and the SHA-1 hash that identifies the commit (463dc4f). The full hash is longer (40 hexadecimal characters), but Git shows a short prefix that is sufficient to uniquely identify the commit in this repository.

```
# ignore all .a files
*.a

# but do track lib.a, even though you're ignoring .a files above
!lib.a

# only ignore the TODO file in the current directory, not subdir/TODD
/TODD

# ignore all files in any directory named build
build/

# ignore doc/notes.txt, but not doc/server/arch.txt
doc/*.txt

# ignore all .pdf files in the doc/ directory and any of its subdirectories
doc/**/*.pdf
```

.gitignore example



More examples at <https://github.com/github/gitignore>



Cloning an existing Git Repository

Cloning a Git Repository

> Use `git clone` to obtain a copy of an existing repository

Cloning retrieves a full copy of the repository, including complete history

```
% git clone <url>
```

```
% git clone <url> <directory-name>
```

Creates a new directory, initializes a .git repository, and checks out the latest version. Repositories can be cloned using different protocols (e.g., HTTPS, SSH).

<Quick Demo>

Let's clone an existing repository:

```
% git clone https://github.com/libgit2/libgit2
```

creates a directory named libgit2, initializes a .git directory inside it, pulls down all the data for that repository, and checks out a working copy of the latest version

Let's quickly change the `README.md` file and play with the following commands:

```
% git status
```

```
% git add README.md
```

```
% git commit -m "message"
```



Commit History

Viewing the Commit History

> Use `git log` to inspect past commits

```
% git log -p -2
```

list the two most recent commits

shows the difference introduced
in each commit

```
% git log --stat -1
```

shows summarized stats for each commit

```
% git log --pretty=oneline
```

shows each commit in a single line
(options: short, full, oneline, format: "%h - %an: %s")

list most recent commits with its SHA-1 checksum, the author's name and email, the date written and the commit message

```
commit b4eb482bc929ade17b0a5c9bc0a0c6e45a09016
Author: bakersdozen123 <taunts_bakers_3r@icloud.com>
Date: Sun Jan 11 18:07:28 2026 -0800

    Fix 'git_index_entry' documentation

    Change from "Documentation/technical/index-format.txt" to "Documentation/gitformat-index.adoc". Fixes #7115.

commit d90800046420c694f93285600f3d325c38ed2691
Merge: 53eca1064 8fe2c189e
Author: Edward Thomson <ethomson@edwardthomson.com>
Date: Sun Jan 11 22:52:34 2026 +0000

    Merge pull request #7190 from libgit2/ethomson/config

    config: Fix potential null value passed to %s

commit 8fe2c189e5549ef2c9d92b2eb8382802bc8c0bc0
Author: Edward Thomson <ethomson@edwardthomson.com>
Date: Sun Jan 11 01:01:21 2026 +0000

    config: consistent error messages
```

```
bnp@Bernardos-MBP libgit2 % git log --stat -1
commit 74d3a927170221cd8c7a608e3190ddee3dcc52f5 (HEAD -> main)
Author: B <Bernardo.Nunes@anu.edu.au>
Date: Sun Jan 25 00:48:34 2026 +1100

    Changed README file to include greetings

README.md | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
```

More at https://git-scm.com/book/en/v2/Git-Basics-Viewing-the-Commit-History#log_options



Commit History

Viewing the Commit History

```
% git log --since=2.weeks
% git log --since="2020-03-25"
% git log --since="3 days 2 minutes ago"
% git log --author="Bernardo"
% git log --author="Bernardo" --grep="fix"
% git log -S "functionName"
```

other similar options are --until, --after, --before

filters the commit history to show only commits made by an author whose name or email matches "Bernardo"

Use -i for case-insensitive

grep filters commit history by commit message content

it only shows commits adding/removing code matching the string

More at https://git-scm.com/book/en/v2/Git-Basics-Viewing-the-Commit-History#log_options



Oops! Fixing Your Last Commit

Rewriting the last commit

> In case you forgot to add files or made a mistake in the commit message, Git provides tools to undo changes. However, some actions rewrite history and cannot be easily undone (be careful!)

```
% git commit --amend
```

→ replaces the most recent commit with a new one. Uses the current staging area as the new commit content. If nothing new is staged, only the commit message is updated

```
% git commit -m "Initial commit"  
% git add <forgotten_file>  
% git commit --amend
```

> **Note that** amending a commit replaces it entirely (*new commit, new hash*); the old commit effectively disappears from history!

Warning: Only amend commits that are **local and not yet pushed**.
Amending pushed commits can cause serious problems for collaborators.



Oops! Fixing Your Last Commit

Unstaging a staged file with Git Reset*

> The command we use to determine the state of the staging area and working directory reminds us how to undo changes to them:

```
$ git add *
$ git status
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)
```

```
renamed:    README.md -> README
modified:   CONTRIBUTING.md
```

```
$ git reset HEAD CONTRIBUTING.md
Unstaged changes after reset:
M       CONTRIBUTING.md
$ git status
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

renamed:    README.md -> README

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

modified:   CONTRIBUTING.md
```

*Note that Git version 2.23.0 onwards, Git will use `git restore` instead of `git reset` for many undo operations. See next slides.



Oops! Fixing Your Last Commit

Discarding Local Changes to a File

> If you decide you no longer want the changes made to a file, you can restore it to the last committed version. Git provides a command for this, but it must be used with care.

```
% git checkout -- <file>
```

> Replaces the file in your working directory with the last committed (or staged) version

> Permanently discards all uncommitted local changes to that file

> This operation is destructive. Any changes that were never committed are lost and cannot be recovered



Oops! Fixing Your Last Commit

Unstaging a staged file with Git Restore (recommended)

> Again, the command we use to determine the state of the staging area and working directory (`git status`) reminds us how to undo changes to them:

```
$ git add *
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   CONTRIBUTING.md
    renamed:    README.md -> README
```

```
$ git restore --staged CONTRIBUTING.md
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    renamed:    README.md -> README
```

```
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   CONTRIBUTING.md
```

The CONTRIBUTING.md file is still modified, but it is no longer staged



Oops! Fixing Your Last Commit

Discarding Local Changes to a File using Git restore

> If you decide you no longer want the changes made to a file, you can restore it to the last committed version. Git provides a command for this, but it must be used with care.

```
% git status (will tell you how to do it!)  
% git restore <file>
```

> Replaces the file in your working directory with the last committed (or staged) version
> Permanently discards all uncommitted local changes to that file

> This command discards all local changes and restores the file to the last staged or committed version. Use it only if you are certain you no longer need those changes!

> Use `git restore --source=<commit id> <file>` to restore the file from a specific commit



Working with Remote Repositories

What are remotes?

- > A remote is another Git repository that your repo can communicate with
- > Remotes are often hosted online (e.g., GitHub, **GitLab**) but can be on any machine or network
- > Remotes are used to share work and collaborate with others

```
% git remote  
% git remote -v
```

} A remote is basically a named reference to another repository
Git remote list remotes and -v show remotes with URLs

- > Remotes can be added following:

```
% git remote add <name> <url>  
% git remote add origin https://github.com/user/repo.git
```

→ Name is a short alias (often origin)
 <url> is where the remote repository is hosted

Example:

```
% git clone https://gitlab.comp.anu.edu.au/uxxxxxxx/comp7710-git-example.git  
% git remote -v
```



Working with Remote Repositories

Example:

```
% git clone https://gitlab.comp.anu.edu.au/uxxxxxxx/comp7710-git-example.git
% git remote -v
```

Fetch and Pull (Getting Changes)

> Fetch updates your local repo with commits from the remote without merging:

```
% git fetch <remote>
```

> Pull fetches and then merges those changes into your current branch:

```
% git pull <remote> <branch>
```

—————→ pull = fetch + merge by default

Push (Sharing Work)

> To send your commits to a remote branch:

```
% git push <remote> <branch>
```

> If pushing a new branch for the first time:

```
% git push --set-upstream origin <branch>
```

—————→ This sets up tracking between your local branch and the remote branch so future pushes/pulls are easier. Clone has already done that for us.



Working with Remote Repositories

Working with someone else's repository: FORK

> A fork is your own copy of someone else's repository, created under your account on a platform like GitLab or GitHub

> What a fork means ...

>> It starts as an **exact copy** of the original repository

>> You have **full permission** to push, commit, and change it (because it is your copy)

>> The **original repository remains unchanged**; *any changes must be proposed via merge/pull requests*

> How to fork?

> Go to our COMP GitLab: <https://gitlab.comp.anu.edu.au/>

> Open the repository

> Click on Fork

> Choose your account/group

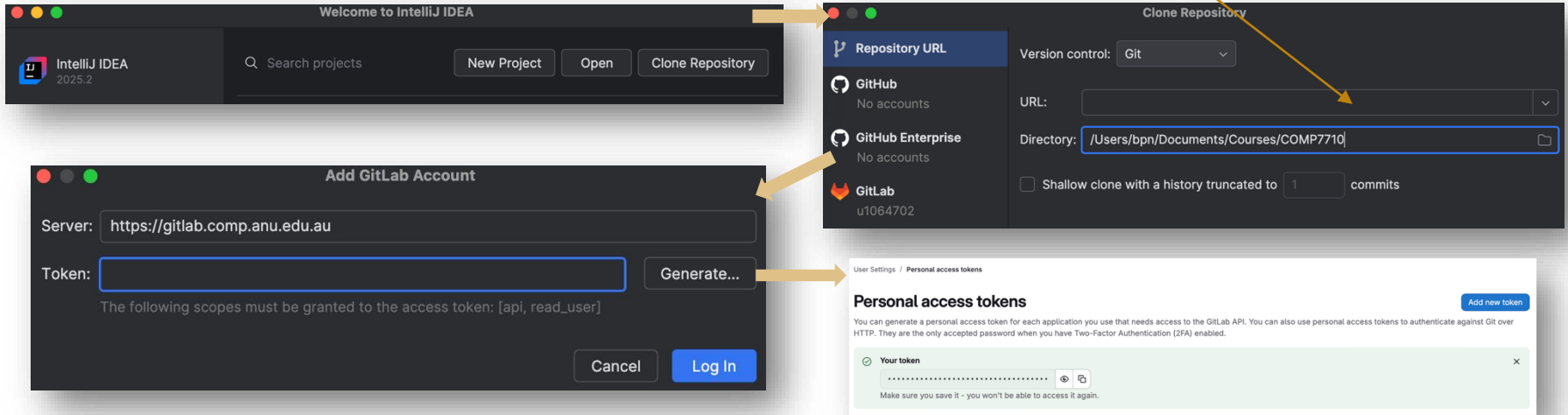
>> You can also fork from the terminal (but not using git commands; you must use GitLab's API or CLI)



Git, GitLab and IntelliJ

How to use VCS in IntelliJ

- > On the GitLab page of your fork, under the Code dropdown, copy the URL under Clone with HTTPS.
- >> It will probably look like <https://gitlab.comp.anu.edu.au/uXXXXXXX/comp7710XXXXX.git>
- > Open IntelliJ, select File > New > Project from Version Control.



Git, GitLab and IntelliJ

<Quick Demo – IntelliJ>



<Exercises>

Which of the following best describes **version control**?

- A. A tool that automatically writes code for you
- B. A system that tracks changes to files over time
- C. A backup system that only stores the latest version
- D. A tool used only for source code files



<Exercises>

Which statement is **true** about **Centralised Version Control Systems (CVCS)**?

- A. Every user has a full copy of the repository history
- B. They work well offline
- C. They have a single point of failure
- D. They are more fault-tolerant than DVCS



<Exercises>

A file that has been changed but not yet added to the staging area is in which state?

- A. Committed
- B. Staged
- C. Modified
- D. Ignored



<Exercises>

What does git fetch do?

- A. Fetches changes and merges them automatically
- B. Deletes local branches
- C. Pushes local commits to the remote
- D. Downloads updates from a remote without merging



<Exercises>

What is a **fork** in GitLab?

- A. A local copy of a repository on your computer
- B. A branch inside the original repository
- C. Your own server-side copy of someone else's repository
- D. A temporary clone that expires



MEME



Thank you



Australian
National
University

TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY)

CRICOS PROVIDER CODE: 00120C